TalkLate

Traduccion en tiempo real con conciencia cultural y emocional

6. Drivers de arquitectura

Trabajo de grado

Julian David Diaz Medina

Diego Alejandro Franco Atehortua

Director: Wider Farid Sanchez Garzon

Version 1.1 - Noviembre de 2026

Fase 3 - Arquitectura

Version: 1.0 (29/10/2026)

Un driver es un requisito que obliga a tomar decisiones de arquitectura. Cada driver tiene una meta medible, la forma de medirla y las decisiones (ADR, *documento 11 (Decisiones de arquitectura (ADR))*) que lo responden. Los dos que definen el producto son DRV-01 (tiempo por turno) y DRV-02 (calidad cultural y emocional).

1. Drivers

ID	Driver	Meta medible	Origen
DRV-01	Tiempo de respuesta por turno	Texto: 3 s o menos (p95). Voz: 5 s o menos, contando transcripcion y audio de salida	RNF-001, vision
DRV-02	Calidad cultural y emocional	80 % de los modismos del glosario con su equivalente cultural; la emocion detectada se escucha en la voz de salida	RNF-002, RNF-010, vision
DRV-03	Privacidad de los datos	0 accesos a datos de otro usuario; mensajes cifrados en reposo; logs sin datos personales; consentimiento y borrado de cuenta (Ley 1581 de 2012)	RNF-003
DRV-04	Crecimiento de idiomas	Agregar un idioma o una region sin cambiar codigo	RNF-004, impacto del Administrador
DRV-05	Continuidad ante fallas de IA	Si falla un motor de traduccion o de voz, el servicio sigue con el siguiente	RNF-005
DRV-06	Costo y abuso	Whisper local sin costo por mensaje; la traduccion (Claude en Vertex) y la voz (ElevenLabs) cuestan por uso, por eso hay limite de 30 mensajes por minuto y 40 voces por minuto por usuario	RNF-006
DRV-07	Una base de codigo para varias plataformas	Mismo codigo para iOS, Android y web	RNF-007
DRV-08	Facilidad de prueba y mantenimiento	Reglas probadas sin internet ni claves; cobertura de 80 % o mas; reglas de dependencia entre capas verificables con dependency-cruiser	RNF-008, RNF-009

2. Como se responde y como se comprueba

Driver	Decisiones	Como se mide	Resultado actual
DRV-01	ADR-002 (REST por turno), ADR-004 (motores en cadena), ADR-014 (voz con Whisper), ADR-019 (voz en la nube), ADR-020 (traduccion con Claude en Vertex)	`npm run medir` (texto) y `npm run medir:voz` (voz) en `backend/`; evento de log `traduccion` con `duracion_ms`	Pendiente: medir con los motores de produccion (Claude en Vertex, Whisper y ElevenLabs) en el equipo de despliegue; Claude y ElevenLabs son llamadas a la nube y su latencia hay que comprobarla

Driver	Decisiones	Como se mide	Resultado actual
DRV-02	ADR-004 (prompt de equivalencia cultural), ADR-014 (entonacion por texto y voz), ADR-015 y ADR-019 (voz con emocion), ADR-020 (transcreacion cultural)	`npm run medir` con un glosario de prueba separado del usado para construir las reglas; prueba de escucha con usuarios	El motor de reglas logra 14 de 15 modismos; el resultado que cuenta para el driver es el del motor de produccion (Claude en Vertex) con un glosario separado, que esta pendiente
DRV-03	ADR-006 (autenticacion), ADR-010 (seguridad y datos personales), ADR-018 (Trivy)	Pruebas de integracion y unitarias de seguridad; escaneo de Trivy en cada pull request y cada semana	Cumple: aislamiento entre usuarios, cifrado en reposo, consentimiento, borrado de cuenta y logs sin datos personales. Ojo: el texto sale a terceros (Google Cloud para traducir y ElevenLabs para la voz); ver *documento 17 (Politica de tratamiento de datos personales)*
DRV-04	ADR-003 (capas), ADR-008 (modelo de dominio)	Prueba del administrador que activa un idioma y agrega una region	Cumple
DRV-05	ADR-004 (Circuit Breaker), ADR-015 (cadena de voz)	`backend/test/motores.test.js`	Cumple de forma parcial: Claude (Vertex) -> reglas, que solo adaptan los modismos que conocen y sin coincidencias responden un error claro; ElevenLabs -> Azure -> OpenAI -> voz del celular (ver ADR-020)
DRV-06	ADR-004, ADR-010, ADR-016	Limites en `src/adaptadores/http/limites.js` y pruebas de seguridad	Cumple
DRV-07	ADR-001 (Expo)	Compilacion web y ejecucion en Expo Go	Cumple en web y en Expo Go
DRV-08	ADR-002, ADR-003, ADR-013 (hexagonal), ADR-017 (piramide de pruebas)	Piramide de pruebas en CI: unitarias, integracion con `npm run coverage`, punta a punta de la API y de la app web; `npm run arch:check` (dependency-cruiser) para las capas	Cumple: 78 unitarias, 29 de integracion (18 de la API contra SQLite y SurrealDB) y 4 de punta a punta; cobertura de lineas de 96.5 %; `arch:check` sin violaciones